

1 Update on Prompt Optimization

Based on our experiments so far, we observe that using feedback from both correct predictions and mistakes generally improves validation performance. However, the best-performing approach to date is surprisingly simple: optimizing the prompt without providing any feedback, i.e., asking an LLM to improve the prompt without showing any examples.

We are not yet confident that this approach is optimal. Further experiments are needed, especially since our task is a binary classification setting where the LLM outputs only **yes/no** without any reasoning.

2 Motivation

After reviewing several related works, it appears that the strongest methods combine multiple optimization components rather than relying on a single strategy. This motivates exploring more structured and hybrid approaches to prompt optimization.

3 My core ideas

1. **Mutation with Prompt Traces.** During mutation, we provide traces of prompt changes from parent prompts. This helps identify which changes improved validation performance and guides future mutations.
2. **Complementary Prompt Crossover.** For a sampled prompt X , instead of random crossover, we select a complementary prompt Y that performs well on cases where X performs poorly, and perform crossover between X and Y .

4 Additional Ideas

- **Grouped Error Feedback.** Inspired by [1], errors can be grouped into categories and provided as batch feedback, similar to batch updates in backpropagation.
- **Instruction + Example Optimization.** As shown in [2], optimizing examples using a simple instruction can outperform instruction-only optimization. Example optimization selects the best K labeled relation–query instances from a pool. Combining instruction and example optimization yields the most robust performance.
- **MCTS-based Search.** Use Monte Carlo Tree Search to balance exploration and exploitation during prompt optimization.

5 Framework Overview

Figure 1 presents the overall framework. Starting from an initial prompt, we first apply instruction optimization, followed by example optimization.

Figure 2 illustrates the mutation strategy, which includes mutation without feedback, mutation with prompt traces, and mutation with explicit feedback.

Figure 3 shows the crossover strategy, where prompts are combined either randomly or using complementary prompt selection.

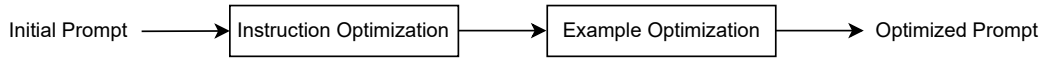


Figure 1: Overview of the approach. Given an initial prompt, we will first to Instruction Optimization followed by Example Optimization.

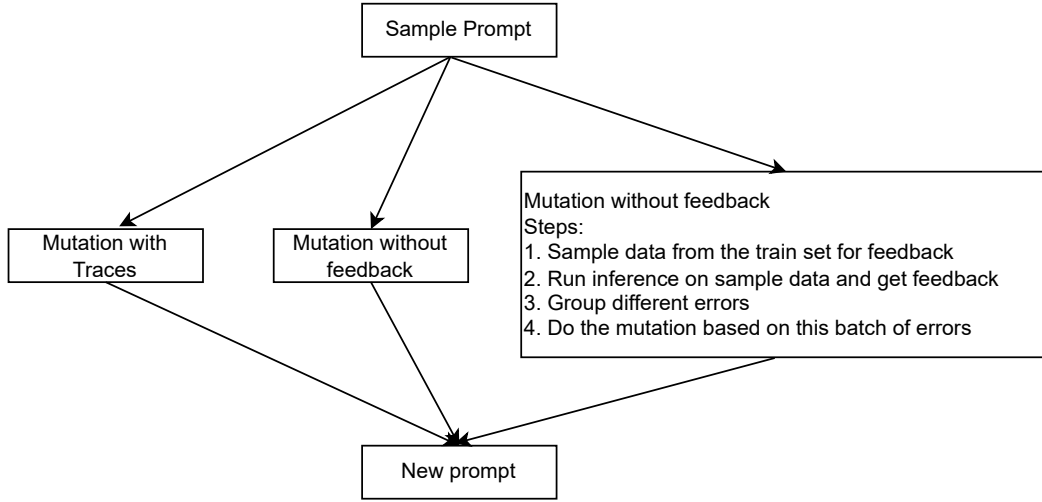


Figure 2: This shows the mutation approach. After we sample the prompt, we have three options to mutate this prompt: Mutate without feedback, Mutate with traces, and Mutate with feedback.

References

- [1] Mayank Singh, Vikas Yadav, and Eduardo Blanco. Error taxonomy-guided prompt optimization, 2026.
- [2] Xingchen Wan, Ruoxi Sun, Hootan Nakhost, and Serkan O Arik. Teach better or show smarter? on instructions and exemplars in automatic prompt optimization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

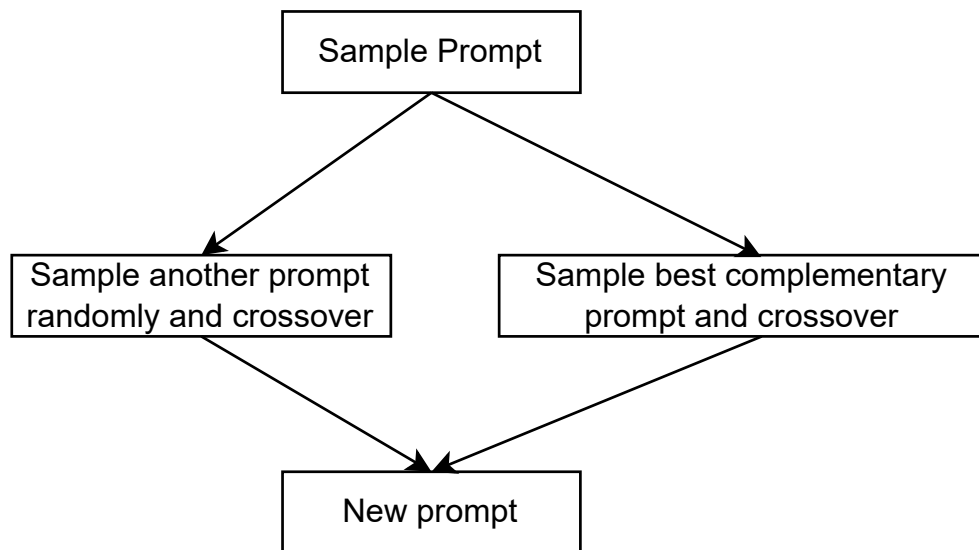


Figure 3: This shows the crossover approach. After we sample the prompt, do crossover with another prompt (selected randomly or select best complimentary prompt).